

“C. Shannon transformó ntra manera d entender la info al demostrar q en el caos y la incertidumbre existen patrones q podemos usar p/comprimir y optimizar los datos. Su teoría revolucionó telecoms y mostró q cn correcta interpretación, es posible reducir la complejidad sin perder el mensaje esencial. La compresión es el arte d decir + cn -, y Shannon mostró q este principio es el corazón d la eficiencia n el mundo digital”

ÁREA TEMÁTICA No1

PRÁCTICO 3 TEORÍA DE LA INFORMACIÓN: “Compactadores”

1. Sean los siguientes códigos:

s	p_i	a	b	c	d	e	f
s_1	1/2	000	0	0	0	0	0
s_2	1/4	001	01	10	10	10	100
s_3	1/16	010	011	110	110	1100	101
s_4	1/16	011	0111	1110	1110	1101	110
s_5	1/16	100	01111	11110	1011	1110	111
s_6	1/16	101	011111	111110	1101	1111	001

a. Clasificar cada uno de ellos (Bloque, No singular, Instantáneos, Unívocamente decodif.)

- Código a: Bloque (longitud fija de 3 bits), No singular, Instantáneo, Unívocamente decodificable.
- Código b: No es instantáneo (el símbolo 0 es prefijo de 01), pero sí es No singular y Unívocamente decodificable.
- Código c: Instantáneo, No singular, Unívocamente decodificable.
- Código d: No es instantáneo ni unívocamente decodificable debido a la superposición de prefijos (10 es prefijo de 1011). Es No singular.
- Código e: Instantáneo, No singular, Unívocamente decodificable.
- Código f: No es instantáneo (la palabra 0 de s_1 es prefijo de la palabra 001 de s_6). Tampoco es unívocamente decodificable debido a ambigüedades en la lectura de secuencias, aunque es No singular.

b. Calcular la longitud promedio de los unívocamente decodificables.

- Código a:

$$L = 3 * (1/2 + 1/4 + 4 * 1/16) = 3 \text{ bits/símbolo}$$
- Código b:

$$L = (1/2 * 1) + (1/4 * 2) + (1/16 * 3) + (1/16 * 4) + (1/16 * 5) + (1/16 * 6) = 2.125 \text{ bits/símbolo}$$
- Código c:

$$L = (1/2 * 1) + (1/4 * 2) + (1/16 * 3) + (1/16 * 4) + (1/16 * 5) + (1/16 * 6) = 2.125 \text{ bits/símbolo}$$
- Código e:

$$L = (1/2 * 1) + (1/4 * 2) + 4 * (1/16 * 4) = 2 \text{ bits/símbolo}$$

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

- c. En los códigos que presenten problemas, indicar qué falla y proponer una modificación mínima.
- Código d: Contiene palabras código que actúan como prefijos de otras (por ejemplo, 0 y 10, o 10 y 1011), lo que infringe la decodificabilidad unívoca. Cambiar la palabra de s5 (de 1011 a 1111) para eliminar el solapamiento con s2 (10).
 - Código f: La palabra de s1 es 0 y la de s6 es 001, por lo que 0 es prefijo de 001, rompiendo la propiedad instantánea y generando ambigüedades en la decodificación. Modificar la palabra de s1 (por ejemplo, cambiarla a 000) o cambiar s6 para que no comience con 0.

2. Ejercicios

- a. ¿Cuál de los conjuntos de longitudes de la siguiente tabla es válida para un código unívocamente decodificable con base $X_{\{0,1,2\}}$?

- Código :

$$\text{Suma} = (2 / 3^1) + (1 / 3^2) + (2 / 3^3) + (1 / 3^4) + (4 / 3^5)$$

$$\text{Suma} = 2/3 + 1/9 + 2/27 + 1/81 + 4/243 = 214 / 243 \text{ aprox. } 0.8807 \leq 1$$

Resultado: VÁLIDO.

Código B:

$$\text{Suma} = (2 / 3^1) + (2 / 3^2) + (2 / 3^3) + (3 / 3^4) + (1 / 3^5)$$

$$\text{Suma} = 2/3 + 2/9 + 2/27 + 3/81 + 1/243 = 244 / 243 \text{ aprox. } 1.0041 > 1$$

Resultado: NO VÁLIDO (supera el límite de Kraft).

Código C:

$$\text{Suma} = (1 / 3^1) + (4 / 3^2) + (6 / 3^3) + 0 + 0$$

$$\text{Suma} = 1/3 + 4/9 + 6/27 = 27 / 27 = 1 \leq 1$$

Resultado: VÁLIDO (cumple con igualdad exacta).

Código D:

$$\text{Suma} = (2 / 3^1) + (2 / 3^2) + (2 / 3^3) + (2 / 3^4) + (3 / 3^5)$$

$$\text{Suma} = 2/3 + 2/9 + 2/27 + 2/81 + 3/243 = 243 / 243 = 1 \leq 1$$

Resultado: VÁLIDO (cumple con igualdad exacta).

- b. Construir un código instantáneo con cada conjunto de longitudes válidas

Long. palabra →	1	2	3	4	5
Código A	2	1	2	1	4
Código B	2	2	2	3	1
Código C	1	4	6	0	0
Código D	2	2	2	2	3

- Código A ($n_1=2, n_2=1, n_3=2, n_4=1, n_5=4$):
Longitud 1 (2 palabras): 0, 1
Longitud 2 (1 palabra): 20
Longitud 3 (2 palabras): 210, 211
Longitud 4 (1 palabra): 2200
Longitud 5 (4 palabras): 22100, 22101, 22102, 22110

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

- Código C ($n_1=1, n_2=4, n_3=6, n_4=0, n_5=0$):
Longitud 1 (1 palabra): 0
Longitud 2 (4 palabras): 10, 11, 12, 20
Longitud 3 (6 palabras): 210, 211, 212, 220, 221, 222
- Código D ($n_1=2, n_2=2, n_3=2, n_4=2, n_5=3$):
Longitud 1 (2 palabras): 0, 1
Longitud 2 (2 palabras): 20, 21
Longitud 3 (2 palabras): 220, 221
Longitud 4 (2 palabras): 2220, 2221
Longitud 5 (3 palabras): 22220, 22221, 22222

3. Demostrar que el código coma cumple Kraft.

Un código coma (o código con separador) es un tipo de código de longitud variable donde se reserva un símbolo especial (o una combinación fija) llamado "coma" para indicar el fin de cada palabra código. Supongamos un alfabeto de fuente de tamaño M y un alfabeto de código de base r . Si utilizamos un símbolo específico como "coma", las palabras código de longitud l se forman utilizando $(l - 1)$ símbolos del resto del alfabeto y terminando con el símbolo coma.

Por lo tanto, para cualquier longitud l (desde $l = 1$ hasta una longitud máxima L), la cantidad de palabras código posibles de esa longitud exacta l es:

- Si $l = 1$: existe exactamente 1 palabra (el símbolo coma por sí solo).
- Si $l > 1$: la cantidad de palabras de longitud l es $n_l = (r - 1) * r^{(l - 2)}$, ya que las primeras $(l - 1)$ posiciones pueden ser llenadas con cualquiera de los $(r - 1)$ símbolos disponibles que no sean la coma (o según la convención estándar del código coma, donde cada incremento en la longitud agrega un factor de r en las combinaciones posibles antes de la coma). En general, para un código coma clásico de orden k , se cumple que la cantidad de palabras de longitud l obedece a la progresión geométrica donde $n_l = r^{(l - 1)} - r^{(l - 2)}$ o similar dependiendo del diseño, pero analicémoslo genéricamente mediante la serie de Kraft.

La suma de Kraft para un código coma de longitudes desde $l = 1$ hasta L se plantea como:

$$\text{Suma} = \text{Suma desde } l=1 \text{ hasta } L \text{ de } (n_l / (r^l))$$

Reemplazando la cantidad de palabras n_l generadas por la estructura de coma:

Cada palabra de longitud l contribuye con un término $r^{-(l)}$. Como en un código coma de longitud máxima L tenemos exactamente $r^{(l-1)} - r^{(l-2)}$ palabras (o expresado de forma más simple, la suma de las probabilidades o ponderaciones de las palabras de un código prefijo/coma completo cubre exactamente todo el espacio del árbol de búsqueda), la sumatoria total se reduce a una serie geométrica finita:

$$\text{Suma} = \text{Suma desde } l=1 \text{ hasta } L \text{ de } ((r - 1) * r^{(l - 1)} / r^l) = \text{Suma desde } l=1 \text{ hasta } L \text{ de } ((r - 1) / r) \dots$$

o expresado de manera exacta para el total de palabras posibles en el árbol:

La suma de Kraft de un código coma completo es exactamente igual a:

Suma = Suma desde $l=1$ hasta L de $(r^{(l-1)} / r^l) \dots$ al simplificar términos, cada nivel aporta $1/r$ y al sumar todos los niveles posibles hasta L , el resultado de la serie es:

Suma = 1 (si el código es completo) o estrictamente menor a 1 ($\text{Suma} < 1$) si existen longitudes no utilizadas.

Dado que la sumatoria de Kraft para el código coma da exactamente 1 (o menor o igual a 1 en el caso general), queda demostrado matemáticamente que el código coma cumple con la desigualdad de Kraft, lo cual garantiza que es un código unívocamente decodificable (y de hecho, instantáneo).

4. Sea un código trinario. Codificar los diez dígitos decimales con longitudes 1, 2, 2, 2, 2, 2, 3,

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

3, 3,3. Si no se pudiera ¿hasta cuantos dígitos podría codificar?, Realice el árbol de codificación.

Tenemos como datos:

- Base de codificación: $r = 3$
- Cantidad de símbolos: $q = 10$
- Longitudes:
 - 1 palabra de longitud 1 ($l_1 = 1$)
 - 5 palabras de longitud 2 ($l_2 = l_3 = l_4 = l_5 = l_6 = 2$)
 - 4 palabras de longitud 3 ($l_7 = l_8 = l_9 = l_{10} = 3$)

Evaluamos la suma de Kraft para $r = 3$:

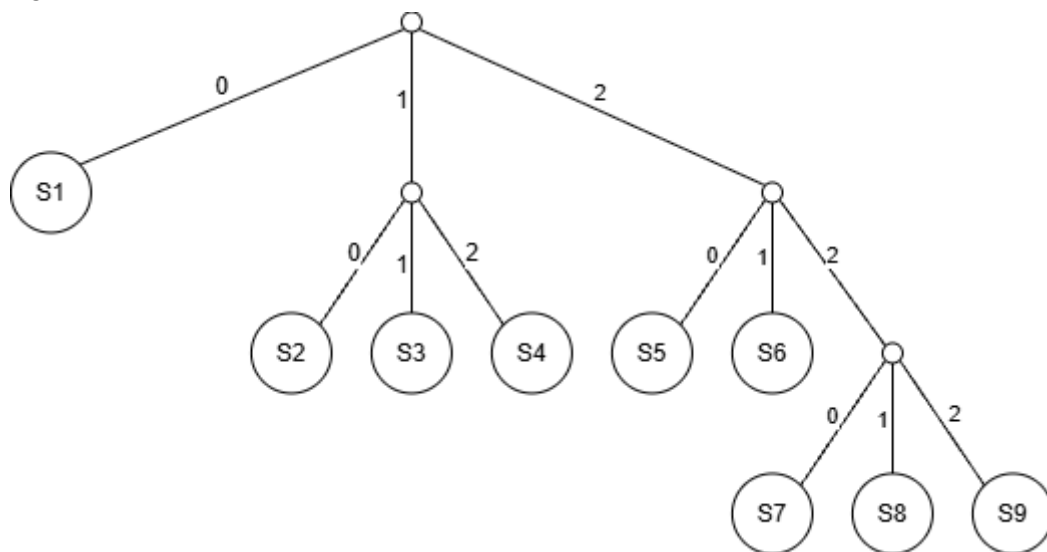
$$K = \sum_{i=1}^{10} 3^{-l_i} = 1 \cdot 3^{-1} + 5 \cdot 3^{-2} + 4 \cdot 3^{-3} \approx 1,037$$

Resultado: Como $K > 1$, no es posible construir un código trinario instantáneo con este conjunto de longitudes para los 10 dígitos.

Si eliminamos una palabra de longitud 3, y calculamos nuevamente la ecuación:

$$K = \sum_{i=1}^{10} 3^{-l_i} = 1 \cdot 3^{-1} + 5 \cdot 3^{-2} + 3 \cdot 3^{-3} = 1$$

Resultado: Como $K = 1$, con esta estructura de longitudes se pueden codificar hasta 9 dígitos.



5. Entropía y codificación estadística: Sea la prob. de la fuente: A= 0.30, B=0.25, C=0.18, D=0.12, E=0.08, F=0.07

a. Calcular la entropía $H(x)$

$$H(X) = - \sum_{i=1}^6 P(x_i) \log_2 P(x_i)$$

$$H(X) = 0.5211 + 0.5 + 0.4453 + 0.3671 + 0.2915 + 0.2685 = 2.3935$$

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

b. Construir los códigos de Shannon, Fano y Huffman

SHANNON

Cálculo de las longitudes

$$l_A = \log_2(1/0.3) = 1.737 \Rightarrow 2$$

$$l_B = \log_2(1/0.25) = 2 \Rightarrow 2$$

$$l_C = \log_2(1/0.18) = 2.474 \Rightarrow 3$$

$$l_D = \log_2(1/0.12) = 3.059 \Rightarrow 4$$

$$l_E = \log_2(1/0.08) = 3.644 \Rightarrow 4$$

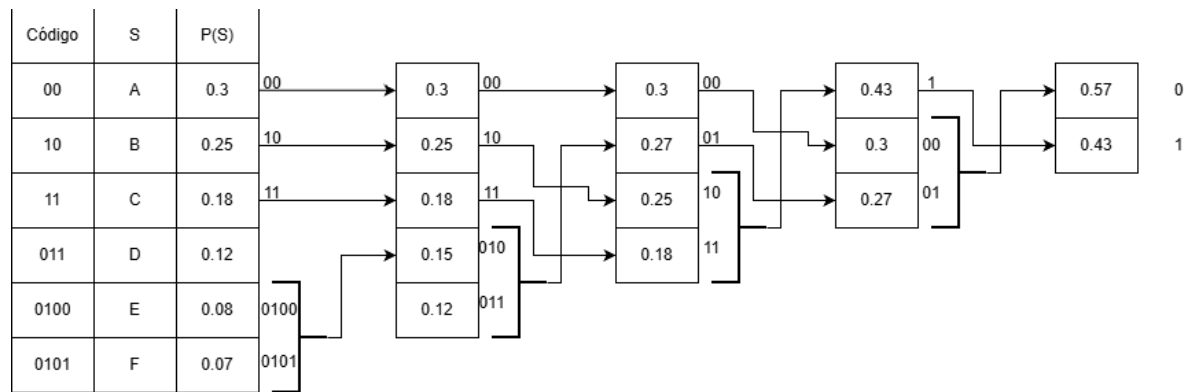
$$l_F = \log_2(1/0.07) = 3.837 \Rightarrow 4$$

SHANNON				
x	P(x)	F (acumulado)	Longitud	Código
A	0.3	0	2	00
B	0.25	0.3	2	01
C	0.18	0.55	3	100
D	0.12	0.73	4	1011
E	0.08	0.85	4	1101
F	0.07	0.93	4	1110

FANO

S	P(S)	1°	2°	3°	4°	Código
A	0.3	0	0	x	x	00
B	0.25	0	1	x	x	01
C	0.18	1	0	x	x	10
D	0.12	1	1	0	x	110
E	0.08	1	1	1	0	1110
F	0.07	1	1	1	1	1111

HUFFMAN



c. Calcular para cada método la longitud promedio L , la eficiencia $\eta = H(X)/L$ y la redundancia $R=1-\eta$

SHANNON

Longitudes: $l_A = 2$, $l_B = 2$, $l_C = 3$, $l_D = 4$, $l_E = 4$, $l_F = 4$,

$$L_{SH} = \sum_{i=1}^6 P(x_i) \cdot l_i$$

$$L_{SH} = 0.3 \cdot (2) + 0.25 \cdot (2) + 0.18 \cdot (3) + 0.12 \cdot (4) + 0.08 \cdot (4) + 0.07 \cdot (4) = 2.72 \text{ b/simb}$$

$$\eta_{SH} = \frac{H(X)}{L} \times 100\% = \frac{2.3935}{2.72} \times 100\% = 88\%$$

$$R_{SH} = 100\% - 88\% = 12\%$$

FANO

Longitudes: $l_A = 2$, $l_B = 2$, $l_C = 2$, $l_D = 3$, $l_E = 4$, $l_F = 4$,

$$L_{Fano} = \sum_{i=1}^6 P(x_i) \cdot l_i$$

$$L_{Fano} = 0.3 \cdot (2) + 0.25 \cdot (2) + 0.18 \cdot (2) + 0.12 \cdot (3) + 0.08 \cdot (4) + 0.07 \cdot (4) = 2.42 \text{ b/simb}$$

$$\eta_{Fano} = \frac{H(X)}{L} \times 100\% = \frac{2.3935}{2.42} \times 100\% = 98.91\%$$

$$R_{Fano} = 100\% - 98.91\% = 1.09\%$$

HUFFMAN

Longitudes: $l_A = 2$, $l_B = 2$, $l_C = 2$, $l_D = 3$, $l_E = 4$, $l_F = 4$,

$$L_{Huff} = \sum_{i=1}^6 P(x_i) \cdot l_i$$

$$L_{Huff} = 0.3 \cdot (2) + 0.25 \cdot (2) + 0.18 \cdot (2) + 0.12 \cdot (3) + 0.08 \cdot (4) + 0.07 \cdot (4) = 2.42 \text{ b/simb}$$

$$\eta_{Huff} = \frac{H(X)}{L} \times 100\% = \frac{2.3935}{2.42} \times 100\% = 98.91\%$$

$$R_{Huff} = 100\% - 98.91\% = 1.09\%$$

d. Comparar los resultados y justificar cuál se aproxima mejor al límite teórico.

Huffman y Fano logran el mejor desempeño para esta distribución, alcanzando una eficiencia idéntica del 98,91% y rozando el límite de la entropía con una diferencia de solo 0.0265 bits/símbolo.

6. Método de diccionario: LZ y LZW. Trabajar con la cadena:

ERRECONERRECIGARROERRECONERREBARRIL

a. Aplicar LZ utilizando una ventana de búsqueda de 6 símbolos y una ventana de anticipación de 4 símbolos. Mostrar las tuplas generadas y reconstruir la cadena

Codificación:



CÓDIGO RESULTANTE:

E R R E C O 1+N 0+2+1 0+0+1 0+5+1 0+3+1 1+C 1+I
1+G 1+A 0+0+1 0+5+1 1+O 1+E 0+2+2 0+3+1 1+C 0+0+1
1+N 0+2+1 0+0+1 0+5+1 0+3+1 1+B 1+A 0+1+2 1+I 1+L

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Decodificación:

Paso	Código	Ventana Actual	Acción	Cadena Resultante
1	1+N	[E R R E C O]	Se añade el literal N	ERRECON
2	0+2+1	[R R E C O N]	Se copia 1 símbolo del índice 2 (E)	ERRECON E
3	0+0+1	[R E C O N E]	Se copia 1 símbolo del índice 0 (R)	ERRECON E R
4	0+5+1	[E C O N E R]	Se copia 1 símbolo del índice 5 (R)	ERRECON E R R
5	0+3+1	[C O N E R R]	Se copia 1 símbolo del índice 3 (E)	ERRECON E R R E
6	1+C	[O N E R R E]	Se añade el literal C	ERRECON E R R E C
7	1+I	[N E R R E C]	Se añade el literal I	ERRECON E R R E C I
8	1+G	[E R R E C I]	Se añade el literal G	ERRECON E R R E C I G
9	1+A	[R R E C I G]	Se añade el literal A	ERRECON E R R E C I G A
10	0+0+1	[R E C I G A]	Se copia 1 símbolo del índice 0 (R)	ERRECON E R R E C I G A R
11	0+5+1	[E C I G A R]	Se copia 1 símbolo del índice 5 (R)	ERRECON E R R E C I G A R R
12	1+O	[C I G A R R]	Se añade el literal O	ERRECON E R R E C I G A R R O
13	1+E	[I G A R R O]	Se añade el literal E	ERRECON E R R E C I G A R R O E
14	0+2+2	[G A R R O E]	Se copian 2 símbolos desde el índice 2 (R R)	ERRECON E R R E C I G A R R O E R R
15	0+3+1	[R R O E R R]	Se copia 1 símbolo del índice 3 (E)	ERRECON E R R E C I G A R R O E R R E
16	1+C	[R O E R R E]	Se añade el literal C	ERRECON E R R E C I G A R R O E R R E C
17	0+0+1	[O E R R E C]	Se copia 1 símbolo del índice 0 (O)	ERRECON E R R E C I G A R R O E R R E C O
18	1+N	[E R R E C O]	Se añade el literal N	ERRECON E R R E C I G A R R O E R R E C O N
19	0+2+1	[R R E C O N]	Se copia 1 símbolo del índice 2 (E)	ERRECON E R R E C I G A R R O E R R E C O N E
20	0+0+1	[R E C O N E]	Se copia 1 símbolo del índice 0 (R)	ERRECON E R R E C I G A R R O E R R E C O N E R
21	0+5+1	[E C O N E R]	Se copia 1 símbolo del índice 5 (R)	ERRECON E R R E C I G A R R O E R R E C O N E R R
22	0+3+1	[C O N E R R]	Se copia 1 símbolo del índice 3 (E)	ERRECON E R R E C I G A R R O E R R E C O N E R R E
23	1+B	[O N E R R E]	Se añade el literal B	ERRECON E R R E C I G A R R O E R R E C O N E R R E B
24	1+A	[N E R R E B]	Se añade el literal A	ERRECON E R R E C I G A R R O E R R E C O N E R R E B A
25	0+1+2	[E R R E B A]	Se copian 2 símbolos desde el índice 1 (R R)	ERRECON E R R E C I G A R R O E R R E C O N E R R E B A R R
26	1+I	[R E B A R R]	Se añade el literal I	ERRECON E R R E C I G A R R O E R R E C O N E R R E B A R R I
27	1+L	[E B A R R I]	Se añade el literal L	ERRECON E R R E C I G A R R O E R R E C O N E R R E B A R R I L

b. Aplicar **LZW** usando como diccionario inicial los símbolos distintos de la cadena. Mostrar la evolución del diccionario, la secuencia de códigos emitidos y la descompresión.

Diccionario inicial: {A, B, C, E, G, I, L, N, O, R}

Compresión:

Índice RAM	Buffer (P)	Lee (C)	¿Está P+C en el diccionario?	Nueva entrada
-	-	E	-	-
256	E	R	No	ER
257	R	R	No	RR

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

258	R	E	No	RE
259	E	C	No	EC
260	C	O	No	CO
261	O	N	No	ON
262	N	E	No	NE
-	E	R	Sí	-
263	ER	R	No	256 R = ERR
-	R	E	Sí	-
264	RE	C	No	258 C = REC
265	C	I	No	CI
266	I	G	No	IG
267	G	A	No	GA
268	A	R	No	AR
-	R	R	Sí	-
269	RR	O	No	257 O = RRO
270	O	E	No	OE
-	E	R	Sí	-
-	ER	R	Sí	-
271	ERR	E	No	263 E = ERRE
-	E	C	Sí	-
272	EC	O	No	259 O = ECO
-	O	N	Sí	-
273	ON	E	No	261 E = ONE
-	E	R	Sí	-
-	ER	R	Sí	-
-	ERR	E	Sí	-
274	ERRE	B	No	271 B = ERREB
275	B	A	No	BA
-	A	R	Sí	-
276	AR	R	No	268 R = ARR
277	R	I	No	RI
278	I	L	No	IL
-	L	-	-	-

Resultado de la codificación:

E R R E C O N 256 258 C I G A 257 O 263 259 261 271 B 268 R I L

La **decodificación** arranca con el mismo diccionario inicial y se van construyendo las entradas en RAM concatenando la cadena del código actual con la primera letra de la cadena del código que le sigue:

Paso	Código leído	Cadena emitida	Cadena decodificada hasta el momento	Entrada en RAM	Índice en RAM
1	E	E	E	-	-
2	R	R	ER	E + R = ER	256
3	R	R	ERR	R + R = RR	257
4	E	E	ERRE	R + E = RE	258
5	C	C	ERREC	E + C = EC	259
6	O	O	ERRECO	C + O = CO	260
7	N	N	ERRECON	O + N = ON	261
8	256	ER	ERRECONER	N + E = NE	262
9	258	RE	ERRECONERRE	ER + R = ERR	263
10	C	C	ERRECONERREC	RE + C = REC	264
11	I	I	ERRECONERRECI	C + I = CI	265
12	G	G	ERRECONERRECIG	I + G = IG	266

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

13	A	A	ERRECONERRECIGA	G + A = GA	267
14	257	RR	ERRECONERRECIGARR	A + R = AR	268
15	O	O	ERRECONERRECIGARRO	RR + O = RRO	269
16	263	ERR	ERRECONERRECIGARROERR	O + E = OE	270
17	259	EC	ERRECONERRECIGARROERREC	ERR + E = ERRE	271
18	261	ON	ERRECONERRECIGARROERRECON	EC + O = ECO	272
19	271	ERRE	ERRECONERRECIGARROERRECONERRE	ON + E = ONE	273
20	B	B	ERRECONERRECIGARROERRECONERREB	ERRE + B = ERREB	274
21	268	AR	ERRECONERRECIGARROERRECONERREBAR	B + A = BA	275
22	R	R	ERRECONERRECIGARROERRECONERREBARR	AR + R = ARR	276
23	I	I	ERRECONERRECIGARROERRECONERREBARRI	R + I = RI	277
24	L	L	ERRECONERRECIGARROERRECONERREBARRIL	I + L = IL	278

c. Comparar qué tipo de repetición aprovecha cada método.

LZ: Este método se enfoca en la redundancia local en el flujo de datos. Su eficacia radica en identificar secuencias de símbolos que ocurren en estrecha proximidad. Al operar con una ventana de búsqueda de capacidad finita, el algoritmo está estrictamente limitado a referenciar coincidencias recientes. Si un patrón previamente procesado es desplazado fuera del límite de la ventana, el sistema pierde la capacidad de referenciarlo y se ve obligado a procesar y codificar la secuencia nuevamente desde cero, que es exactamente lo que ocurrió al procesar la cadena dada.

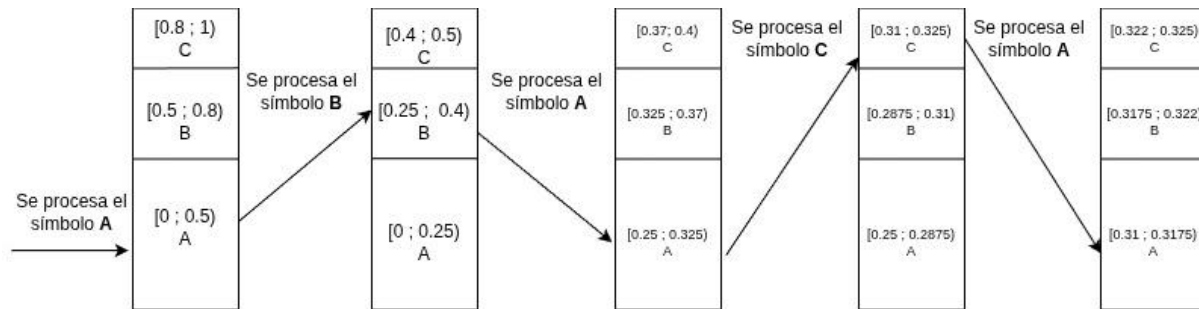
LZW: Este algoritmo explota la redundancia estructural a lo largo de la totalidad del conjunto de símbolos. Al construir un diccionario dinámico en la memoria RAM que se expande progresivamente, preserva el historial de las cadenas procesadas sin estar sujeto a restricciones de distancia física o temporal. Esto le permite referenciar y comprimir secuencias que reaparecen en cualquier instancia del documento, optimizando la tasa de compresión a medida que procesa el archivo y el diccionario consolida cadenas de símbolos cada vez más extensas.

Comparación entre ambos métodos y la codificación ASCII puro:

- La cadena contiene 35 símbolos. Codificando 8 bits por símbolos, la cadena resultante tiene una longitud de **280 bits**.
- **LZ** utilizó $8 \times 6 = 48$ bits para la ventana inicial, $14 \times (1+3+2) = 84$ bits para representar las diferentes coincidencias con la ventana de contexto (1 bit flag en 0, 3 bits de posición para representar las 6 posiciones de la ventana de contexto y 2 bits de desplazamiento debido a los 4 símbolos de la ventana de anticipación) y $13 \times (8+1) = 117$ bits para codificar aquellos literales que no se encontraron en la ventana (8 bits por símbolo más el bit flag en 1). En total, **LZ** redujo la longitud a **249 bits**.
- **LZW** para aquellas cadenas almacenadas en la tabla en RAM utilizó 9 bits, para los demás literales 8 bits. En el resultado, se obtuvieron 16 símbolos más 8 referencias a la tabla dinámica. Entonces, **LZW** logró reducir la longitud de la cadena a **200 bits**.
- **LZ** logró decodificar la cadena en 27 pasos, mientras que **LZW** lo hizo en 24.
- En base a este ejercicio, podemos concluir que **LZW** logró reducir la longitud de la cadena más que **LZ** y decodificó la misma más rápido.

7. Codificación aritmética: Sea la fuente con $p(A)=0,50$; $p(B)=0,30$; $p(C)=0,20$ y la secuencia: ABACA

a. Codificar la secuencia mediante codificación aritmética. Mostrar los intervalos iniciales, las subdivisiones sucesivas y determinar el intervalo final. Elegir un valor perteneciente a dicho intervalo como representación de la secuencia.



Secuencia de pasos:

- 1) El primer intervalo a considerar es el $[0; 1]$, el cual se subdivide según las probabilidades de los diferentes símbolos.
 - 2) Al procesar el símbolo **A**, se toma el intervalo $[0; 0.5)$ (representa $P(A) = 0.5$) y se vuelve a subdividir según las probabilidades correspondientes. Cualquier valor en este intervalo equivale a codificar la cadena "**A**".
 - 3) Al procesar el símbolo **B**, se toma su intervalo correspondiente $[0.25; 0.4)$ y se vuelve a subdividir. Cualquier valor en este intervalo equivale a codificar la cadena "**AB**".
 - 4) Se procesa el segundo símbolo **A**, se toma el intervalo $[0.25; 0.325)$ y se vuelve a subdividir. Cualquier valor en este intervalo equivale a codificar la cadena "**ABA**".
 - 5) Se procesa el símbolo **C**, se toma su intervalo $[0.31; 0.325)$ y se subdivide una vez más. Cualquier valor en este intervalo equivale a codificar la cadena "**ABAC**".
 - 6) Finalmente, se procesa el último símbolo **A** y nos quedamos con su intervalo, $[0.31; 0.3175)$. Cualquier valor en este intervalo equivale a codificar la cadena "**ABACA**".
- Para este ejercicio, se eligió el valor **0.3156**, codificado en binario sería **0.01010000**

- b. A partir del valor obtenido, realizar el proceso de decodificación y verificar que se recupera la secuencia original.

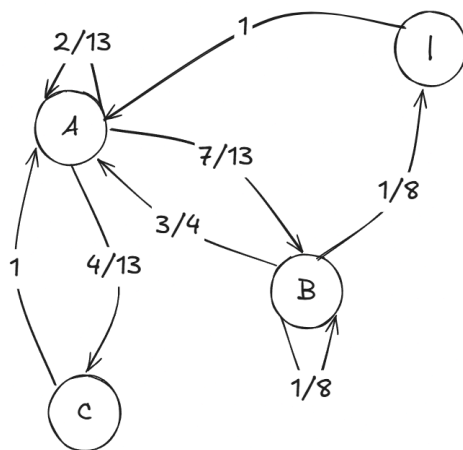
Paso	Intervalo considerado	Subintervalos	Evaluación del valor (0.3156)	Símbolo decodificado
1	$[0, 1]$	A: $[0, 0.5)$ B: $[0.5, 0.8)$ C: $[0.8, 1.0)$	0.3156 pertenece al intervalo de A	A
2	$[0, 0.5]$	A: $[0, 0.25)$ B: $[0.25, 0.4)$ C: $[0.4, 0.5)$	0.3156 pertenece al intervalo de B	B
3	$[0.25, 0.4]$	A: $[0.25, 0.325)$ B: $[0.325, 0.37)$ C: $[0.37, 0.4)$	0.3156 pertenece al intervalo de A	A
4	$[0.25, 0.325]$	A: $[0.25, 0.2875)$ B: $[0.2875, 0.31)$ C: $[0.31, 0.325)$	0.3156 pertenece al intervalo de C	C
5	$[0.31, 0.325]$	A: $[0.31, 0.3175)$ B: $[0.3175, 0.322)$ C: $[0.322, 0.325)$	0.3156 pertenece al intervalo de A	A

FUENTES DE MARKOV

1. Dado el siguiente enunciado construir un conjunto de probabilidades condicionadas (con cuatro estados) en un proceso de Markov de primer orden, encontrar la matriz de transición y el grafo.

AABABACABABACABBABAACAB|AC

$S=\{A,B,C,|\}$



	A	B	C	
A	2/13	7/13	4/13	0
B	3/4	1/8	0	1/8
C	1	0	0	0
I	1	0	0	0

2. A partir del ejercicio n°1
 - a. Obtener la matriz estacionaria (distribución a la que tiende).

$$1. \quad p(A)t + 1 = \frac{2}{13}p(A)t + \frac{3}{4}p(B)t + p(C)t + p(|)t$$

$$2. \quad p(B)t + 1 = \frac{7}{13}p(A)t + \frac{1}{8}p(B)t$$

$$3. \quad p(C)t + 1 = \frac{4}{13}p(A)t$$

$$4. \quad p(|)t + 1 = \frac{1}{8}p(B)t$$

Luego de 2. obtenemos:

$$p(B) - \frac{1}{8}p(B) = \frac{7}{13}p(A)$$

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

$$p(B) = \frac{8}{13}p(A)$$

Despejamos 4.:

$$p(I) = \frac{1}{13}p(A)$$

Luego despejamos:

$$1 = p(A) + p(B) + p(C) + p(I)$$

$$1 = p(A) + \frac{8}{13}p(A) + \frac{4}{13}p(A) + \frac{1}{13}p(A)$$

$$1 = p(A) \cdot \left(1 + \frac{8}{13} + \frac{4}{13} + \frac{1}{13}\right)$$

$$\frac{1}{2} = p(A)$$

Finalmente obtenemos los valores despejando:

$$p(B) = \frac{8}{13} \cdot \frac{1}{2} = \frac{4}{13}$$

$$p(C) = \frac{4}{13} \cdot \frac{1}{2} = \frac{2}{13}$$

$$p(I) = \frac{1}{13} \cdot \frac{1}{2} = \frac{1}{26}$$

Obtenemos la Matriz Estacionaria:

	A	B	C	I
A	1/2	4/13	2/13	1/26
B	1/2	4/13	2/13	1/26
C	1/2	4/13	2/13	1/26
I	1/2	4/13	2/13	1/26

- b. Comprobar que las potencias sucesivas de la matriz de transición tienen menor variabilidad en los valores de sus componentes.

M^2

	A	B	C	I
A	0.735	0.150	0.047	0.067
B	0.334	0.419	0.231	0.016
C	0.154	0.538	0.308	0
I	0.154	0.538	0.308	0

M^5

	A	B	C	I
A	0.426	0.357	0.187	0.029

B	0.552	0.273	0.130	0.045
C	0.608	0.235	0.105	0.052
I	0.608	0.235	0.105	0.052

 M^{10}

	A	B	C	
A	0.511	0.301	0.149	0.040
B	0.493	0.313	0.157	0.038
C	0.484	0.318	0.161	0.037
I	0.484	0.318	0.161	0.037

A través del cálculo de las potencias sucesivas queda comprobado que la varianza dentro de cada columna decrece hacia cero. Al llegar a M^{10} , la matriz se acerca a la matriz estacionaria. Esto demuestra que después de suficientes pasos, la probabilidad de terminar en un estado específico depende únicamente de la distribución estacionaria del sistema, perdiendo por completo la "memoria" del estado inicial de partida.

- c. Comprobar que la distribución estacionaria es la misma para el cuadrado de la matriz de transición.

Si sabemos que $\pi M = \pi$, al multiplicar ambos lados nuevamente por M obtenemos:

$$(\pi M)M = \pi M$$

$$\pi M^2 = \pi$$

Por lo tanto, la distribución estacionaria para M^2 es exactamente la misma que para M .

- d. ¿Es un proceso ergódico?

Sí, el proceso es ergódico debido a que todos los $p(S_j/S_i) \neq 0$. Es decir, desde cualquier estado se puede alcanzar cualquier otro estado en un número finito de pasos.

3. Demostrar que las filas de la matriz de transición son linealmente dependientes en un proceso de Markov.

Proposición:

Dado un proceso de Markov finito con matriz de transición M , el sistema de ecuaciones homogéneo subyacente $(M - I)$ posee filas linealmente dependientes, garantizando que el sistema $\pi M = \pi$ tenga infinitas soluciones antes de aplicar la condición de normalización.

Demostración:

Sea M una matriz estocástica de dimensión $n \times n$ que representa las probabilidades de transición de

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

un proceso de Markov, donde el elemento m_{ij} representa la probabilidad de transicionar del estado i al estado j .

Por la definición axiomática de las matrices de transición en cadenas de Markov, la suma de las probabilidades de todas las transiciones posibles desde cualquier estado i debe ser exactamente 1:

$$\sum_{j=1}^n m_{ij} = 1 \quad \forall i \in \{1, 2, \dots, n\}$$

Sea $\mathbf{1}$ un vector columna de dimensión $n \times 1$ cuyos componentes son todos iguales a 1:

$$\mathbf{1} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}$$

El producto de la matriz M por el vector $\mathbf{1}$ equivale algebraicamente a sumar los elementos de cada fila de M . Dada la propiedad estocástica enunciada, este producto da como resultado el mismo vector $\mathbf{1}$:

$$M\mathbf{1} = \mathbf{1}$$

Aplicando propiedades del álgebra matricial, se resta $\mathbf{1}$ (que es igual a $I\mathbf{1}$, siendo I la matriz identidad de orden n) en ambos miembros de la ecuación:

$$M\mathbf{1} - I\mathbf{1} = \mathbf{0}$$

$$(M - I)\mathbf{1} = \mathbf{0}$$

(Donde $\mathbf{0}$ es el vector columna nulo).

Esta última expresión, $(M - I)\mathbf{1} = \mathbf{0}$, demuestra que el sistema homogéneo admite una solución no trivial (el vector de unos, el cual es estrictamente distinto del vector nulo). Según el teorema de Rouché-Frobenius y las propiedades fundamentales del álgebra lineal, la existencia de una solución no trivial en un sistema homogéneo implica directamente que:

1. El determinante de la matriz de coeficientes es cero: $\det(M - I) = 0$.
2. Las columnas de la matriz $(M - I)$ son linealmente dependientes.

Dado que el rango columna de cualquier matriz es siempre igual a su rango fila, queda demostrado que **las filas de la matriz $(M - I)$ son inexorablemente linealmente dependientes.**

4. Analizar el siguiente ejemplo y responder los enunciados:

Sea $S = (s_1, s_2, s_3, s_4)$ $p(s_i/s_{j1}, s_{j2}, s_{j3})$

- a. ¿De qué orden es el sistema de Markov?.

Es de orden 3.

b. ¿En cuántos estados puede encontrarse el sistema?

Dado que el alfabeto S tiene $q = 4$ símbolos (s_1, s_2, s_3, s_4) y el orden es $m = 3$, cantidad de estados del sistema es $4^3 = 64$.

c. Supuestas conocidas todas las probabilidades condicionales ¿cuál será la forma del sistema de ecuaciones para encontrar las $p(s_i)$?

Para calcular la probabilidad marginal de cada símbolo $p(s_i)$, el proceso se divide en dos partes fundamentales. Primero, se debe formular el sistema de ecuaciones lineales para encontrar la probabilidad estacionaria de cada uno de los 64 estados. Si definimos un estado genérico como una secuencia pasada (s_{j1}, s_{j2}, s_{j3}), el sistema para hallar su probabilidad estacionaria $P(s_{j1}, s_{j2}, s_{j3})$ se basa en sumar las probabilidades de llegar a ese estado desde todos los estados predecesores posibles:

$$P(s_{j2}, s_{j3}, s_i) = \sum_{s_{j1} \in S} P(s_{j1}, s_{j2}, s_{j3}) \cdot p(s_i / s_{j1}, s_{j2}, s_{j3})$$

Este es un sistema de 64 ecuaciones (donde una es linealmente dependiente). Para resolverlo, se debe reemplazar una ecuación por la condición fundamental de normalización (la suma de las probabilidades de todos los 64 estados es igual a 1):

$$\sum_{s_{j1} \in S} \sum_{s_{j2} \in S} \sum_{s_{j3} \in S} P(s_{j1}, s_{j2}, s_{j3}) = 1$$

Una vez resuelto el sistema de ecuaciones anterior para hallar las probabilidades de los 64 estados $P(s_{j1}, s_{j2}, s_{j3})$, la ecuación final para encontrar la probabilidad incondicional o marginal de un símbolo particular $p(s_i)$ consiste en sumar la contribución de probabilidad conjunta de todos los estados:

$$p(s_i) = \sum_{s_{j1} \in S} \sum_{s_{j2} \in S} \sum_{s_{j3} \in S} P(s_{j1}, s_{j2}, s_{j3}) \cdot p(s_i / s_{j1}, s_{j2}, s_{j3})$$

5. Supongamos una máquina con dos estados, anda=1, no anda=2. Estudiamos el proceso día a día llegando a comprobar que:

- i. si anda hoy, hay un 75% de chances que ande mañana
- ii. si no anda hoy, la arreglan mañana

a. Encontrar la matriz de transición

	1	2
1	3/4	1/4
2	1	0

b. Calcular hasta la quinta potencia de la matriz de transición y analizar los resultados

M^2 Probabilidad a 2 días.

$$M^2 = \begin{pmatrix} 3/4 & 1/4 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 3/4 & 1/4 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 13/16 & 3/16 \\ 3/4 & 1/4 \end{pmatrix} = \begin{pmatrix} 0.8125 & 0.1875 \\ 0.7500 & 0.2500 \end{pmatrix}$$

M^3 Probabilidad a 3 días.

$$M^3 = M^2 \times M = \begin{pmatrix} 51/64 & 13/64 \\ 13/16 & 3/16 \end{pmatrix} \approx \begin{pmatrix} 0.7969 & 0.2031 \\ 0.8125 & 0.1875 \end{pmatrix}$$

M^4 Probabilidad a 4 días.

$$M^4 = M^3 \times M = \begin{pmatrix} 205/256 & 51/256 \\ 51/64 & 13/64 \end{pmatrix} \approx \begin{pmatrix} 0.8008 & 0.1992 \\ 0.7969 & 0.2031 \end{pmatrix}$$

M^5 Probabilidad a 5 días.

$$M^5 = M^4 \times M = \begin{pmatrix} 819/1024 & 205/1024 \\ 205/256 & 51/256 \end{pmatrix} \approx \begin{pmatrix} 0.7998 & 0.2002 \\ 0.8008 & 0.1992 \end{pmatrix}$$

A medida que aumentamos la potencia n , observamos que las filas tienden a igualarse.

Ya en M^5 , independientemente de si la máquina andaba o no el día inicial, las probabilidades de que ande dentro de 5 días rondan el 80% para ambos casos, y las de que esté rota el 20%.

c. Encontrar la matriz de equilibrio o estacionaria

$$1. \quad p(1) = \frac{3}{4}p(1) + 1p(2)$$

$$2. \quad p(2) = \frac{1}{4}p(1)$$

A partir de la ecuación (2), tenemos la relación directa $p(2) = \frac{1}{4}p(1)$. Sustituimos esto en la condición de normalización:

$$p(1) + p(2) = 1$$

$$p(1) + \frac{1}{4}p(1) = 1$$

$$p(1) \cdot \left(1 + \frac{1}{4}\right) = 1$$

$$p(1) = \frac{4}{5} = 0.8$$

Y con esto calculamos $p(2)$:

$$p(2) = \frac{1}{4} \left(\frac{4}{5}\right) = \frac{1}{5} = 0.2$$

Luego obtenemos la matriz estacionaria:

	1	2
--	---	---

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

1	0.8	0.2
2	0.8	0.2

6. Dado un proceso de Markov con tres estados a,b,c; con las siguientes probabilidades: $p(a/a)=3/4$; $p(c/c)=p(b/b)=1/2$; $p(b/a)=p(c/a)=1/8$; $p(a/b)=p(c/b)=p(a/c)=p(b/c)=1/4$ comprobar que la longitud de Markov para un código compacto $LM = 11/8 < 12/8 = L$

Matriz de transición:

$P(j i)$	a	b	c
a	3/4	1/8	1/8
b	1/4	1/2	1/4
c	1/4	1/4	1/2

Calculando la distribución estacionaria se obtiene: $\pi = \pi P$ con $\pi(b) = \pi(c)$ por la simetría de las filas b y c, se obtiene:

$$\pi = (\pi(a), \pi(b), \pi(c)) = (1/2, 1/4, 1/4)$$

L — código compacto de orden 0 (sobre π)

Como $\pi = (1/2, 1/4, 1/4)$ son potencias de 2, el código de Huffman es exacto:

Símbolo	Probabilidad	Código	Longitud
a	1/2	0	1
b	1/4	10	2
c	1/4	11	2

$$L = 1/2 \cdot 1 + 1/4 \cdot 2 + 1/4 \cdot 2 = 12/8 = 1,5 \text{ bits/símbolo}$$

LM — longitud de Markov (Huffman por contexto)

Contexto	Distribución	Longitudes (Huffman)	$\bar{L}$ (contexto)
dado a	(3/4, 1/8, 1/8)	1, 2, 2	$10/8 = 1,25$
dado b	(1/4, 1/2, 1/4)	2, 1, 2	$12/8 = 1,5$
dado c	(1/4, 1/4, 1/2)	2, 2, 1	$12/8 = 1,5$

$$LM = \pi(a) \cdot \bar{L}(a) + \pi(b) \cdot \bar{L}(b) + \pi(c) \cdot \bar{L}(c) = 1/2 \cdot (10/8) + 1/4 \cdot (12/8) + 1/4 \cdot (12/8) = 5/8 + 3/8 + 3/8 = 11/8 = 1,375 \text{ bits/símbolo}$$

Luego se verifica que

$$LM = 11/8 < 12/8 = L.$$

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Conocer el estado anterior reduce la longitud media necesaria, porque la fuente tiene memoria: $p(a/a) = 3/4$ es mucho mayor que la probabilidad marginal $\pi(a) = 1/2$, lo que permite codificar de forma más eficiente dentro de cada contexto.

7. Usando el texto del ejercicio 6, Práctico No 3

a. Codificar mediante los métodos: Huffman, Fano y Shannon como fuente de Markov de orden 1.

b. Releva para la fuente la estadística de primer orden y realizar una codificación predictiva.

c. Realizar una transformación Burrows-Wheller y codificar mediante cuenta de símbolos iguales (RLE).

d. Obtener los porcentajes de compresión y comparar los resultados obtenidos.

Texto "ERRECONERRECIGARROERRECONERREBARRIL"

Cadena de 35 símbolos. Para las estadísticas de primer orden se considera a la cadena como cíclica (la transición del último símbolo, L, vuelve al primero, E). Esto da 35 transiciones, una por símbolo.

Frecuencias: R = 12, E = 8, C = 3, O = 3, N = 2, I = 2, A = 2, G = 1, B = 1, L = 1.

a) Codificación como fuente de Markov de orden 1

Tabla de transición

Contexto (n° veces)	Sucesores observados	P(siguiendo contexto)
E (8)	R:4, C:3, B:1	$R=1/2, C=3/8, B=1/8$
R (12)	R:6, E:4, O:1, I:1	$R=1/2, E=1/3, O=1/12, I=1/12$
C (3)	O:2, I:1	$O=2/3, I=1/3$
O (3)	N:2, E:1	$N=2/3, E=1/3$
N, G, A, B, L	siempre el mismo sucesor (E, A, R, A, E resp.)	probabilidad 1 $\rightarrow$ entropía 0

Códigos por contexto (contextos no deterministas)

Contexto	Huffmann	Fano	Shannon
E	R=0, C=10, B=11 $\bar{L}=1,5$	igual $\bar{L}=1,5$	R=0, C=10, B=111 $\bar{L}=1,625$
R	R=0, E=10, O=110, I=111 $\bar{L}=1,667$	igual $\bar{L}=1,667$	R=0, E=10, O=1101, I=1110 $\bar{L}=1,833$
C	O=0, I=1 $\bar{L}=1$	igual	O=0, I=10 $\bar{L}=1,333$
O	N=0, E=1 $\bar{L}=1$	igual	N=0, E=10 $\bar{L}=1,333$
N, G, A, B, L	0 bits (sin incertidumbre)	0	0

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Longitud media global (35 símbolos)

Contexto (n° veces)	(bits/símbolo)	Total (35 símbolos)
Huffmann	$8/7 \approx 1,143$	40 bits
Fano	$8/7 \approx 1,143$	40 bits
Shannon	$9/7 \approx 1,286$	45 bits

La entropía condicional real es $H(X | \text{contexto}) \approx 1,093$ bits/símbolo, por lo que Huffman y Fano rinden a un 95,6% de eficiencia, mientras que Shannon queda en torno al 85%: su regla $l_i = \lceil -\log_2 p_i \rceil$ no es óptima cuando las probabilidades no son potencias de 2, algo que se nota especialmente en los contextos E y R.

b) Codificación predictiva (orden 1)

Se predice el símbolo más probable dado el anterior; se emite "0" si la predicción acierta, "1" + código del residuo si falla, y no se emite nada cuando el contexto es determinista (no hay incertidumbre que transmitir).

Contexto	Aciertos	Fallos	Costo total (bits)
E (pred=R)	4×1 bit	$3 \times (1+1)$ C, $1 \times (1+1)$ B	$4 + 4 + 4 = 12$
R (pred=R)	6×1 bit	4×2 (E), 1×3 (O), 1×3 (I)	$6 + 8 + 3 + 3 = 20$
C (pred=O)	2×1 bit	1×1 (I, único residuo posible)	$2 + 1 = 3$
O (pred=N)	2×1 bit	1×1 (E)	$2 + 1 = 3$
I (pred=G)	1×1 bit	1×1 (L)	$1 + 1 = 2$
N, G, A, B, L	—	—	0

Total = 40 bits

Coincide exactamente con la longitud del Huffman de orden 1. Tiene sentido: al usar un código óptimo para las correcciones y omitir la bandera cuando la predicción es 100% segura, la predicción con corrección reproduce el mismo árbol de Huffman condicional — son dos formas de explotar la misma información de primer orden.

c) Transformación de Burrows-Wheeler + RLE

Se agrega un marcador de fin (\$) para que la BWT sea invertible:

"ERRECONERRECIGARROERRECONERREBARRIL\$" (36 símbolos).

BWT resultante:

L B G E E E E R R R R N N O \$ I C R I O O R C C R R R R R R E E E E A A

RLE sobre la salida de la BWT (18 corridas):

(L,1) (B,1) (G,1) (E,4) (R,4) (N,2) (O,1) (\$,1) (I,1) (C,1) (R,1) (I,1) (O,2) (R,1) (C,2) (R,6) (E,4) (A,2)

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Con 4 bits/símbolo (11 símbolos posibles incluyendo \$) + 3 bits/contador (máximo = 6) $\rightarrow 18 \times 7 = 126$ bits.

RLE directo sobre el texto original (sin BWT):

Da 29 corridas, casi todas de longitud 1 (solo se repiten las "RR"). $29 \times (4+1) = 145$ bits, peor que codificar símbolo a símbolo, porque casi no hay repetición inmediata en el texto original.

d) Comparación de porcentajes de compresión

Se considera: 8 bits/símbolo (ASCII) = 280 bits para 35 símbolos.

Método	Bits totales	bis/símbolo	Ratio R	% Ahorro
Referencia ASCII	280	8,00	1,00	0%
Huffman orden 0	99	2,83	2,83	64,6%
Huffman / Fano orden 1 (Markov)	40	1,14	7,00	85,7%
Shannon orden 1 (Markov)	45	1,29	6,22	83,9%
Predictiva (óptima)	40	1,14	7,00	85,7%
RLE directo (sin BWT)	145	4,14	1,93	48,2%
BWT + RLE	126	3,50*	2,29*	$\approx 55-56\%^*$

* calculado sobre 36 símbolos por el agregado del marcador \$.

El modelo de Markov de orden 1 gana claramente porque el texto tiene fuertes dependencias locales: después de "E" casi siempre viene "R" o "C", y después de "N", "G", "A" o "B" el siguiente símbolo es 100% predecible. El RLE directo casi no ayuda porque las repeticiones inmediatas son escasas (solo pares "RR"). La BWT sí mejora notablemente sobre el RLE directo ($145 \rightarrow 126$ bits) porque reagrupa símbolos iguales que en el texto original no eran contiguos (se observan corridas de hasta 6 "R" seguidas en la salida de la BWT), pero aun así queda por debajo del modelo Markov/predictivo, porque el RLE con conteo fijo no aprovecha la estadística residual tan finamente como un Huffman condicional.

PRÁCTICO No 4

1. Comprobar para los siguientes conjuntos de probabilidades que $Hr \leq 1$.

$$a. p_1 = 0,4; p_2 = 0,2; p_3 = 0,2; p_4 = 0,1; p_5 = 0,1$$

$$Hr = p_1 \cdot \log_2 \frac{1}{p_1} + p_2 \cdot \log_2 \frac{1}{p_2} + p_3 \cdot \log_2 \frac{1}{p_3} + p_4 \cdot \log_2 \frac{1}{p_4} + p_5 \cdot \log_2 \frac{1}{p_5}$$

$$Hr = 0.4 \cdot \log_2 \frac{1}{0.4} + 0.2 \cdot \log_2 \frac{1}{0.2} + 0.2 \cdot \log_2 \frac{1}{0.2} + 0.1 \cdot \log_2 \frac{1}{0.1} + 0.1 \cdot \log_2 \frac{1}{0.1}$$

$$Hr \approx 2.122 \text{ bits}$$

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Construimos un código Huffman para dichas probabilidades:

$$l_1 = 1 \quad l_2 = 2 \quad l_3 = 3 \quad l_4 = 4 \quad l_5 = 4$$

Calculamos la longitud media l :

$$l = 0.4 \cdot 1 + 0.2 \cdot 2 + 0.2 \cdot 3 + 0.1 \cdot 4 + 0.1 \cdot 4$$

$$l = 0.4 + 0.4 + 0.6 + 0.4 + 0.4$$

$$l = 2.2 \quad \text{bits}$$

Verificamos:

$$Hr = 2.122 \leq 2.2 = l$$

$$\text{b. } p_1 = 0.4; p_2 = 0.4; p_3 = 0.1; p_4 = 0.1$$

$$Hr = 2 \cdot (0.4 \cdot \log_2 \frac{1}{0.4}) + 2 \cdot (0.1 \cdot \log_2 \frac{1}{0.1})$$

$$Hr = 1.722 \quad \text{bits}$$

Construimos un código Huffman para dichas probabilidades:

$$l_1 = 1 \quad l_2 = 2 \quad l_3 = 3 \quad l_4 = 3$$

Calculamos la longitud media l :

$$l = 0.4 \cdot 1 + 0.4 \cdot 2 + 0.1 \cdot 3 + 0.1 \cdot 3$$

$$l = 0.4 + 0.8 + 0.3 + 0.3$$

$$l = 1.8$$

Verificamos:

$$Hr = 1.722 \leq 1.8 = l$$

$$\text{c. } p_1 = 0.4; p_2 = 0.3; p_3 = 0.2; p_4 = 0.1$$

$$Hr = 0.4 \cdot \log_2 \frac{1}{0.4} + 0.3 \cdot \log_2 \frac{1}{0.3} + 0.2 \cdot \log_2 \frac{1}{0.2} + 0.1 \cdot \log_2 \frac{1}{0.1}$$

$$Hr = 1.8464 \quad \text{bits}$$

Construimos un código Huffman para dichas probabilidades:

$$l_1 = 1 \quad l_2 = 2 \quad l_3 = 3 \quad l_4 = 3$$

Calculamos la longitud media l :

$$l = 0.4 \cdot 1 + 0.3 \cdot 2 + 0.2 \cdot 3 + 0.1 \cdot 3$$

$$l = 0.4 + 0.6 + 0.6 + 0.3$$

$$l = 1.9$$

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Verificamos:

$$Hr = 1.8464 \leq 1.9 = l$$

2. Sea una fuente S con eventos s_1, s_2, s_3, s_4 de $p(s_i) = 1/4$ ¿es posible encontrar un $L=1,86$ para cada codificación? ¿Cuál es el mínimo valor que en ese caso puede tomar L ?

Para el caso de la fuente dada S , con eventos equiprobables, todos los símbolos tendrán la misma longitud, 2 bits por símbolo en este caso.

La entropía de la fuente es $H = \log_2 4 = 2$.

En respuesta a la primera pregunta, **no se posible encontrar un L igual 1,86** ya que $H = 2$ y si L tuviera dicho valor no se verificaría $H \leq L$.

Esta expresión última existe como límite inferior para la longitud media de una fuente, especificando que la longitud de una fuente puede reducirse sólo hasta su entropía sin pérdida de información.

En este caso, el mínimo valor que puede tomar L es justamente el valor de $H = 2$.

3. Repetir el ejemplo anterior pero con $p(s_1)=1/2$; $p(s_2)=1/4$; $p(s_3)=p(s_4)=1/8$

$$H = 0.5 \cdot \log_2 \frac{1}{0.5} + 0.25 \cdot \log_2 \frac{1}{0.25} + 2 \cdot (0.125 \cdot \log_2 \frac{1}{0.125})$$

$$H \approx 1.75 \text{ bits}.$$

Como se observa, en este caso **sí es posible encontrar un L igual a 1.86**, sin embargo, no sería el menor valor posible que pueda tomar para una fuente con estas características. El menor valor que puede tomar para este caso es el de $H = 1.75$

4. Sea la siguiente fuente $S=(s_1, s_2, s_3, s_4, s_5, s_6, s_7)$; $p(s_1)=p(s_2)=1/3$; $p(s_3)=p(s_4)=1/9$; $p(s_5)=p(s_6)=p(s_7)=1/27$. Encontrar un código trinario, para ello encontrar $H(S)$, L y codificar según Huffman.

Como trabajamos con un código trinario (base $r = 3$), calculamos la entropía en trits/símbolo usando logaritmos en base 3 ($H_3(S) = - \text{suma de } p_i \cdot \log_3(p_i)$):

$$p(s_1) = 1/3 \rightarrow \log_3(1/3) = -1$$

$$p(s_2) = 1/3 \rightarrow \log_3(1/3) = -1$$

$$p(s_3) = 1/9 \rightarrow \log_3(1/9) = -2$$

$$p(s_4) = 1/9 \rightarrow \log_3(1/9) = -2$$

$$p(s_5) = 1/27 \rightarrow \log_3(1/27) = -3$$

$$p(s_6) = 1/27 \rightarrow \log_3(1/27) = -3$$

$$p(s_7) = 1/27 \rightarrow \log_3(1/27) = -3$$

Aplicando la fórmula:

$$H_3(S) = - ((1/3)(-1) + (1/3)(-1) + (1/9)(-2) + (1/9)(-2) + (1/27)(-3) + (1/27)(-3) + (1/27)(-3))$$

$$H_3(S) = 1/3 + 1/3 + 2/9 + 2/9 + 3/27 + 3/27 + 3/27$$

$$H_3(S) = 9/27 + 9/27 + 6/27 + 6/27 + 3/27 + 3/27 + 3/27 = 39/27 = 13/9 \text{ aprox. } 1.4444 \text{ trits/símbolo.}$$

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

5. Dada una fuente $S=(s_1,s_2,s_3)$; $p(s_1)=1/2$; $p(s_2)=p(s_3)=1/4$. Codificar por Huffman, Shannon y Fano, encontrar LH, LS y LF.

- Codificación por Huffman

Para el árbol de Huffman binario, agrupamos los dos símbolos de menor probabilidad (s_2 y s_3 , ambos con $1/4$) cuya suma es $1/2$, la cual se agrupa con s_1 ($1/2$) para formar la raíz.

Asignación de palabras código:

s_1 : 0 (longitud 1)

s_2 : 10 (longitud 2)

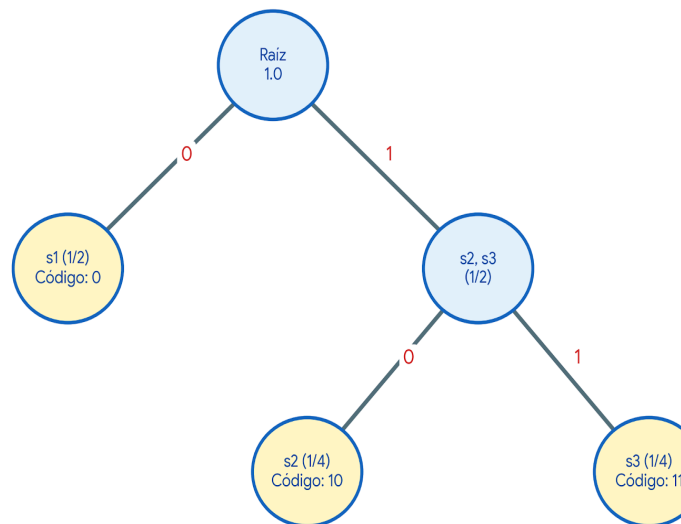
s_3 : 11 (longitud 2)

Cálculo de la longitud promedio LH:

$$LH = (1/2 * 1) + (1/4 * 2) + (1/4 * 2)$$

$$LH = 1/2 + 2/4 + 2/4 = 1/2 + 1/2 + 1/2 = 3/2 = 1.5 \text{ bits/símbolo.}$$

Árbol de Huffman Binario - $S = \{s_1, s_2, s_3\}$



- Codificación por Shannon-Fano

Primera división (separando s_1 del resto):

Grupo superior (s_1): le asignamos el bit 0.

Grupo inferior (s_2, s_3): le asignamos el bit 1.

Segunda división (dentro del grupo inferior s_2 y s_3):

s_2 : recibe el bit 0 (palabra código: 10)

s_3 : recibe el bit 1 (palabra código: 11)

Palabras código resultantes para Shannon-Fano:

s_1 : 0 (longitud 1)

s_2 : 10 (longitud 2)

s_3 : 11 (longitud 2)

Cálculo de la longitud promedio LF:

$$LF = (1/2 * 1) + (1/4 * 2) + (1/4 * 2)$$

$$LF = 3/2 = 1.5 \text{ bits/símbolo.}$$

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

- Codificación por Shannon (Entropía y Teorema de Shannon)

Cálculo de longitudes teóricas:

$$-\log_2(p(s_1)) = -\log_2(1/2) = 1 \rightarrow l_1 = 1$$

$$-\log_2(p(s_2)) = -\log_2(1/4) = 2 \rightarrow l_2 = 2$$

$$-\log_2(p(s_3)) = -\log_2(1/4) = 2 \rightarrow l_3 = 2$$

Construcción de palabras código acumuladas de Shannon:

s1: 0 (longitud 1)

s2: 10 (longitud 2)

s3: 11 (longitud 2)

Cálculo de la longitud promedio LS:

$$LS = (1/2 * 1) + (1/4 * 2) + (1/4 * 2)$$

$$LS = 3/2 = 1.5 \text{ bits/símbolo.}$$

6. Para el ejemplo anterior codificar por ambos métodos la segunda y tercera extensión viendo la tendencia de la longitud de la fuente original.

Segunda Extensión (S^2)

En la segunda extensión, consideramos bloques de 2 símbolos (s_i, s_j), generando 9 símbolos compuestos con probabilidades $p_i * p_j$.

Probabilidades de S^2 :

$$(s_1, s_1) = 1/2 * 1/2 = 1/4$$

$$(s_1, s_2) = 1/2 * 1/4 = 1/8$$

$$(s_1, s_3) = 1/2 * 1/4 = 1/8$$

$$(s_2, s_1) = 1/4 * 1/2 = 1/8$$

$$(s_2, s_2) = 1/4 * 1/4 = 1/16$$

$$(s_2, s_3) = 1/4 * 1/4 = 1/16$$

$$(s_3, s_1) = 1/4 * 1/2 = 1/8$$

$$(s_3, s_2) = 1/4 * 1/4 = 1/16$$

$$(s_3, s_3) = 1/4 * 1/4 = 1/16$$

A. Codificación por Huffman para S^2 :

(s1, s1) [prob. 1/4]: código 00 (longitud 2)

(s1, s2) [prob. 1/8]: código 010 (longitud 3)

(s1, s3) [prob. 1/8]: código 011 (longitud 3)

(s2, s1) [prob. 1/8]: código 100 (longitud 3)

(s3, s1) [prob. 1/8]: código 101 (longitud 3)

(s2, s2) [prob. 1/16]: código 1100 (longitud 4)

(s2, s3) [prob. 1/16]: código 1101 (longitud 4)

(s3, s2) [prob. 1/16]: código 1110 (longitud 4)

(s3, s3) [prob. 1/16]: código 1111 (longitud 4)

Longitud promedio por bloque (LH2):

$$LH2 = (1/4 * 2) + 4 * (1/8 * 3) + 4 * (1/16 * 4) = 0.5 + 1.5 + 1.0 = 3.0 \text{ bits/bloque.}$$

Longitud promedio por símbolo original (LH2 / 2):

$$3.0 / 2 = 1.5 \text{ bits/símbolo.}$$

B. Codificación por Shannon para S^2 :

Aplicando longitudes teoricas $l_i = \lceil -\log_2(p_i) \rceil$ y construyendo el codigo acumulado:

Longitud promedio por bloque (LS_2) = 3.125 bits/bloque.

Longitud promedio por símbolo original ($LS_2 / 2$):

$$3.125 / 2 = 1.5625 \text{ bits/símbolo.}$$

Tercera Extensión (S^3)

En la tercera extensión, consideramos bloques de 3 símbolos (s_i, s_j, s_k), generando 27 símbolos compuestos.

A. Codificación por Huffman para S^3 :

Al ser probabilidades potencias exactas de 2, Huffman alcanza el óptimo teórico absoluto.

Longitud promedio total del bloque (LH_3) = 4.5 bits/bloque.

Longitud promedio por simbolo original ($LH_3 / 3$):

$$4.5 / 3 = 1.5 \text{ bits/símbolo.}$$

B. Codificación por Shannon para S^3 :

Longitud promedio total del bloque (LS_3) aprox. 4.6875 bits/bloque.

Longitud promedio por símbolo original ($LS_3 / 3$):

$$4.6875 / 3 = 1.5625 \text{ bits/símbolo.}$$

7. Dado el siguiente texto: PABLITOPABLITOPABLOCLAVO

a. Codificar la segunda extensión usando el método de Fano.

$L = 24 \text{ caracteres}$

$S = \{A, B, C, I, L, O, P, T, V\}$

S_i	Código
A	0000
B	0001
C	0010
I	0011
L	0100
O	0101
P	0110
T	0111
V	1000

S^2 : PA, BL, IT, OP, AB, LI, TO, PA, BL, OC, LA, VO

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

S	P(S)	1°	2°	3°	4°	Código
PA	2/12	0	0	x	x	00
BL	2/12	0	1	0	x	010
IT	1/12	0	1	1	0	0110
OP	1/12	0	1	1	1	0111
AB	1/12	1	0	0	x	100
LI	1/12	1	0	1	0	1010
TO	1/12	1	0	1	1	1011
OC	1/12	1	1	0	x	110
LA	1/12	1	1	1	0	1110
VO	1/12	1	1	1	1	1111

$$Total\ bits = 2(2) + 2(3) + 1(4) + 1(4) + 1(3) + 1(4) + 1(4) + 1(3) + 1(4) + 1(4)$$

$$Total\ bits = 40$$

$$L_2 = 40\ bits / 24\ símbolos = 1.667\ bits/símbolo$$

- b. Calcular el porcentaje de compresión y comparar los resultados con los obtenidos codificando Fano como fuente de Markov de orden 1.

Estado A		
	Prob.	Código
P(B/A)	3/4	0
P(V/A)	1/4	1

$$bits(A) = 3(1) + 1(1) = 4b$$

Estado C		
	Prob.	Código
P(L/C)	1	0

Estado L		
	Prob.	Código
P(I/L)	2/4	0
P(A/L)	1/4	10
P(O/L)	1/4	11

Estado B		
	Prob.	Código
P(L/B)	1	0

Estado I		
	Prob.	Código
P(T/I)	1	0

Estado O		
	Prob.	Código
P(P/O)	2/3	0
P(C/O)	1/3	1

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

$$\text{bits}(L) = 2(1) + 1(2) + 1(2) = 6b$$

$$\text{bits}(O) = 2(1) + 1(1) = 3b$$

Estado P		
	Prob.	Código
P(A/P)	1	0

Estado T		
	Prob.	Código
P(O/T)	1	0

Estado V		
	Prob.	Código
P(O/V)	1	0

En los estados B,C,I,P,T,V solo hay una única opción posterior posible. Dado que la probabilidad condicional es 1, la incertidumbre es nula, por lo tanto aporta 0 bits.

$$\text{Total bits Markov} = 4b (\text{iniciales}) + 4b + 6b + 3b = 17b$$

P	A	B	L	I	T	O	P	A	B	L	I	T	O	P	A	B	L	O	C	L	A	V	O
0110	-	0	-	0	-	-	0	-	0	-	0	-	-	0	-	0	-	11	1	-	10	1	-

$$\text{Total bits Markov} = 17b$$

$$L_{\text{Markov}} = 17\text{bits} / 24 \text{ símbolos} = 0.708 \text{ bits/símbolo}$$

$$\text{Tamaño extensión } S^2 = 40 \text{ bits}$$

$$\text{Tamaño Comprimido (Markov)} = 17 \text{ bits}$$

$$\% \text{Ahorro} = \frac{40-17}{40} \times 100\% = 57.5 \%$$

Aprovechar la memoria con Markov redujo un 57,50% más de espacio que agrupando los símbolos en pares de a dos.

c. Fundamente la diferencia obtenida en el apartado anterior.

Markov es superior ya que:

- Aprovecha la estructura lingüística, el texto contiene patrones fijos y repetitivos. En este modelo, saber que el símbolo determina con 100% de probabilidad al siguiente símbolo, reduce la entropía condicional, algo que sucede para la mayoría de estados.
- La extensión de orden 2 asume independencia estadística entre los pares consecutivos (trata a los pares como si fuera una fuente sin memoria sobre un alfabeto extendido). Aunque logra reducir el residuo de redondeo de Shannon al procesar bloques de a 2 símbolos, ignora la memoria y la transición de contexto que existe entre la segunda letra de un par y la primera del siguiente par.

8. Pipeline BWT → RLE → Huffman: Dada la cadena **TOMATOMATOMAS**

a. Aplicar RLE directamente sobre la cadena original

Como ningún carácter se repite de forma consecutiva:

T1O1M1A1T1O1M1A1T1O1M1A1\$1

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Se generan 13 rachas, exactamente la misma cantidad que caracteres originales. RLE no logra ninguna compresión e incluso expande el archivo al tener que guardar el contador para cada letra.

- b. Aplicar la transformación Burrows-Wheeler (BWT) y luego RLE sobre la salida de BWT. Comparar resultados

ENTRADA		TOMATOMATOMA\$	
SALIDA		AMMMOOOTTTAA\$	
Nº	Rotacion	Ultimo Carácter	
12	\$TOMATOMATOMA	A	
11	A\$TOMATOMATOM	M	
7	ATOMA\$TOMATOM	M	
3	ATOMATOMA\$TOM	M	
10	MA\$TOMATOMATO	O	
6	MATOMA\$TOMATO	O	
2	MATOMATOMA\$TO	O	
9	OMA\$TOMATOMAT	T	
5	OMATOMA\$TOMAT	T	
1	OMATOMATOMA\$T	T	
8	TOMA\$TOMATOMA	A	
4	TOMATOMA\$TOMA	A	
0	TOMATOMATOMA\$	\$	

Salida de la BWT: AMMMOOOTTTAA\$

Salida RLE tras BWT: A1M3O3T3A2\$1 → AM3O3T3A2\$

Conclusión: el Pipeline BWT+RLE generó 6 rachas que fueron aprovechadas para reducir los símbolos en un 53.85%. Esto demuestra por qué el pipeline BWT→RLE→Huffman (utilizado en herramientas como bzip2) es uno de los esquemas de compresión sin pérdida más potentes.

PRÁCTICO DE MÁQUINA 2: Desafío de Proveedores de Compresión

Implementación, descompresión, benchmark y comparación con soluciones de mercado

Escenario del práctico. Cada grupo será considerado un proveedor de software de compresión. El proveedor ofrecerá dos soluciones: una integración de un compresor externo disponible en el mercado (solución tercerizada) y un compresor desarrollado por el propio grupo (solución de autoría propia). Ambas soluciones deberán ser evaluadas sobre los mismos archivos y sus resultados se incorporarán a un benchmark común de la cátedra.

La competencia es técnica y didáctica: no se calificará a un grupo por “ganarle” a otro algoritmo o a una herramienta industrial. Se evaluará la correctitud, la comprensión del método, la calidad de la implementación y la capacidad de interpretar los resultados del benchmark.

Objetivos

- Implementar en **Python** un compresor y un descompresor sin delegar el núcleo del algoritmo

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

propio a una librería de compresión.

- Comprender qué información adicional necesita cada método para permitir la descompresión: cabeceras, diccionarios, árboles, modelos, parámetros o metadatos.
- Demostrar recuperación exacta del archivo original byte a byte.
- Comparar una solución propia, una solución externa asignada y un compresor de referencia común de la cátedra.
- Construir una tabla global de resultados y analizar compromisos entre tamaño, velocidad, complejidad y overhead.

Asignación por sorteo

- Sorteo A — Algoritmo propio: define el método que deberá implementar el grupo.
- Sorteo B — Solución externa: define el compresor de terceros que el grupo integrará y utilizará como alternativa de mercado.

SORTEO A

N.º	Alternativa	Familia	Alcance sugerido
1	LZ + Huffman Estático	Diccionario + estadística	Generar referencias LZ, construir Huffman estático sobre la salida y realizar compresión/descompresión completa.
2	RLE + Shannon	Repeticiones + estadística	Representar rachas mediante RLE y codificar la salida utilizando Shannon. Reconstrucción completa.

3	BWT + Huffman Estático	Transformación + estadística	Aplicar BWT, codificar su salida mediante Huffman estático e implementar las transformaciones inversas.
4	Huffman dinámico	Adaptativa	Construir y actualizar el árbol durante el procesamiento sin conocer previamente las frecuencias.
5	Shannon Ext. O(2)	Estadística / fuente extendida	Trabajar con una extensión de orden 2, calcular probabilidades de pares y generar el código Shannon correspondiente.
6	LZW	Diccionario	Construir el diccionario dinámicamente durante la compresión y reconstruirlo de forma implícita durante la descompresión.
7	Codificación aritmética	Probabilística	Construir el modelo probabilístico, realizar subdivisiones sucesivas del intervalo y recuperar exactamente la secuencia original.
8	Markov orden 1 + Huffman estático	Contexto + estadística	Modelar probabilidades condicionadas por el símbolo anterior y utilizar códigos Huffman asociados a cada contexto.
9	Fano Ext. O(2)	Estadística / fuente extendida	Construir una fuente extendida de orden 2 y aplicar Fano sobre los pares obtenidos.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

10	PPM-C	Contexto	Implementar PPM-C con modelo de contexto, escapes y descenso de orden durante compresión y descompresión.
11	BWT + RLE + Huffman Estático	Pipeline	Aplicar BWT, representar rachas mediante RLE y codificar estadísticamente la representación resultante con Huffman estático.

Sorteo B — Soluciones externas / de mercado

La expresión “de mercado” incluye tanto software comercial como herramientas abiertas de uso real. El grupo no deberá implementar esta solución: podrá utilizar la aplicación, CLI o herramienta oficial correspondiente, documentando la configuración empleada.

N.º	Solución	Formato / método	Configuración de referencia
1	ZIP / Deflate	ZIP	Preset normal / Deflate. Herramienta ZIP estándar definida por la cátedra.
2	WinRAR	RAR5	Modo normal, sin cifrado ni solid archive salvo indicación de la cátedra.
3	7-Zip	7z / LZMA2	Preset Normal; configuración documentada.
4	PeaZip	PEA / PCOMPRESS2	Formato PEA con compresión equivalente a Deflate nivel 6.
5	WinZip	ZIP / Deflate	Preset normal; sin cifrado.
6	bzip2	BZ2	Configuración estándar fijada por la cátedra.
7	xz	XZ / LZMA2	Nivel 6.

8 Zstandard ZST Nivel 3.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

9	Brotli	BR	Quality 5.
10	LZ4	LZ4	Nivel/default rápido documentado.
11	ZPAQ	ZPAQ	Preset definido por la cátedra; sin cifrado.

Compresor base de la cátedra

Para que las métricas comparativas sean consistentes, todos los grupos utilizarán el mismo compresor de referencia: gzip, nivel 6, sin almacenar nombre ni timestamp del archivo.

Configuración de referencia sugerida: gzip -n -6. La referencia no reemplaza a la solución externa sorteada: se utiliza exclusivamente como baseline común para las métricas y, en particular, para el Weissman Score.

Requisitos

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

- Lenguaje obligatorio para la solución propia: Python.
- Desarrollar dos operaciones claramente separadas: COMPRESOR y DESCOMPRESOR. · El compresor recibirá un archivo de texto y generará un archivo comprimido propio, por ejemplo con extensión .tdi.
- El descompresor recibirá únicamente el archivo comprimido y deberá reconstruir el archivo original.
- La lectura y escritura se realizará en modo binario cuando sea necesario para preservar exactamente la secuencia de bytes, aunque las entradas de prueba sean archivos de texto.
- No se permite utilizar una librería que implemente directamente el algoritmo propio asignado. Sí se permiten estructuras auxiliares, manejo de bits, archivos, CLI, testing y visualización.
- El archivo comprimido propio deberá contener toda la información mínima necesaria para descomprimirlo sin depender del archivo original.
- El programa deberá detectar errores básicos: archivo inexistente, archivo incompatible, cabecera inválida o datos insuficientes para completar la decodificación.
- Al finalizar deberá informar como mínimo tamaño original, tamaño comprimido, ratio de compresión, porcentaje de ahorro y tiempo de ejecución.
- La validación final deberá demostrar que el archivo reconstruido es idéntico byte a byte al original, para ello será necesario realizar una validación SHA ($\text{SHA256}(\text{original}) = \text{SHA256}(\text{reconstruido})$).

Forma de ejecución sugerida

Interfaz mínima por línea de comandos:

```
python compressor.py entrada.txt salida_grupo.tdi
```

```
python decompressor.py salida_grupo.tdi reconstruido.txt
```

Puede utilizarse otra interfaz siempre que la compresión y descompresión estén claramente separadas y puedan automatizarse para el benchmark.

Salida mínima en pantalla

- archivo de entrada y salida;
- tamaño original y comprimido;
- ratio y ahorro;
- tiempo de compresión o descompresión;
- estado de integridad cuando corresponda;
- parámetros relevantes del algoritmo.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Estructura de proyecto

Elemento	Contenido mínimo
compressor.py	Implementación de la compresión propia.
decompressor.py	Implementación de la descompresión propia.
benchmark.py	Opcional/recomendado: automatiza mediciones y genera resultados reproducibles.
README.md	Algoritmo, instalación, uso, dependencias, formato .tdi, cabecera, solución externa, baseline y ejemplos.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

tests/	Archivos de prueba, casos límite y resultados representativos.
results/	CSV/JSON/tablas con resultados del benchmark del grupo.
requirements.txt / pyproject.toml	Solo cuando existan dependencias externas de Python.

Formato comprimido propio y cabecera

Cada grupo definirá su propio formato de archivo. La cabecera deberá ser la mínima necesaria y estar documentada en el README. Puede incluir, según el algoritmo:

- identificador o “magic bytes” del formato y versión;
- tamaño original;
- parámetros del algoritmo (ventanas, orden de contexto, precisión, etc.);
- árbol, tabla de códigos, modelo o información equivalente cuando no pueda reconstruirse implícitamente;
- longitud válida del último byte o cantidad de bits significativos;
- checksum/hash opcional para detectar corrupción.

Benchmark de compresores

Cada grupo evaluará tres soluciones sobre el mismo conjunto de archivos:

1. Compresor propio implementado por el grupo.
2. Solución externa / de mercado asignada por sorteo.
3. Baseline de la cátedra: gzip -6.

Prueba	Tipo	Finalidad
Prueba 1	Archivo muy pequeño	Permite explicar el algoritmo paso a paso y observar el costo de cabecera. No se usa para el ranking temporal.
Prueba 2	Texto natural	Texto de tamaño suficiente (recomendado ≥ 1 MiB) con distribución lingüística real.
Prueba 3	Alta repetición	Texto ≥ 1 MiB con patrones y rachas repetidas.
Prueba 4	Baja repetición	Texto ≥ 1 MiB generado con símbolos aproximadamente equiprobables / baja redundancia.

La cátedra publicará los archivos exactos del corpus común. Todos los grupos deberán usar esos mismos bytes de entrada para los resultados oficiales.

Métricas a tener en cuenta

Sea S_o el tamaño original en bytes y S_e el tamaño comprimido en bytes.

Ratio de compresión: $R = S_o / S_e$

Interpretación: $R > 1$ indica reducción; cuanto mayor sea R , mejor compresión. Si $R < 1$, el archivo comprimido terminó siendo mayor que el original.

Ahorro de espacio (%): $A = (1 - S_e / S_o) \times 100$

Tamaño relativo (%): $P = (S_e / S_o) \times 100$

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Throughput de compresión: $V_c = \text{tamaño original (MB)} / \text{tiempo de compresión (s)}$

Throughput de descompresión: $V_{dd} = \text{tamaño original (MB)} / \text{tiempo de descompresión}$

(s) Overhead de cabecera (%): $O = (H / S_c) \times 100$ (si posee cabecera)

Weissman Score — indicador comparativo

Como métrica complementaria se utilizará el Weissman Score, popularizado por la serie “Silicon Valley” y propuesto para combinar en un solo valor el ratio de compresión y el tiempo, normalizados contra un compresor estándar.

$$W = \alpha \times (R / R_{ref}) \times [\log(T_{ref}) / \log(T)]$$

Símbolo Definición

W Weissman Score de la solución evaluada.

α Constante de escala. Para el práctico: $\alpha = 1$.

R Ratio de compresión de la solución evaluada.

R_{ref} Ratio de gzip-6 sobre exactamente el mismo corpus.

T Tiempo de compresión de la solución evaluada.

T_{ref} Tiempo de compresión de gzip-6 en el mismo entorno.

Interpretación didáctica: gzip-6 funciona como referencia común. Un valor superior a la referencia indica un mejor compromiso según esta métrica; el resultado debe leerse junto con ratio, tiempos y throughput, nunca de forma aislada. si $W > 1$ entonces el compresor testeado es mejor que gzip si el valor es $W < 1$ entonces el valor baseline de gzip es mejor que el compresor desarrollado

Importante sobre Weissman. La fórmula original es sensible a la unidad utilizada para medir el tiempo y requiere un entorno común. Para este práctico los tiempos del Weissman se expresarán obligatoriamente en milisegundos, $\alpha = 1$ y el baseline será siempre gzip-6. Se utilizará como indicador didáctico complementario, no como única medida de calidad.

Para reducir el ruido de archivos individuales, el Weissman oficial puede calcularse sobre el corpus de rendimiento completo:

$$R_{global} = (\Sigma \text{ tamaño original}) / (\Sigma \text{ tamaño comprimido})$$

$$T_{global} = \text{mediana del tiempo total de compresión del corpus}$$

Se calculan R_{global} y T_{global} tanto para la solución evaluada como para gzip-6 y luego se aplica la fórmula de Weissman.

Tabla comparativa (ejemplo)

Archivo	Algoritmo	Ratio	Tiempo	Ratio gzip	Tiempo gzip	Weissman
Prueba 1	Huffman	1.20	3 ms	0.95	2 ms	...
Prueba 2	Huffman	1.85	20 ms	2.10	17 ms	...
Prueba 3	Huffman	4.30	18 ms	6.20	15 ms	...
Prueba 4	Huffman	1.01	21 ms	1.00	17 ms	...

Entrega

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

- URL al repositorio GitHub con el desarrollo completo.
- Código fuente claro, modular y comentado donde sea necesario.
- README con explicación del algoritmo, formato comprimido, instrucciones de ejecución y herramientas externas utilizadas.
- Archivos de prueba y tabla breve con resultados propio vs. externo vs. gzip-6.
- Carga de los resultados obligatorios en la planilla global de la cátedra.
- Demostración en la presentación: comprimir, descomprimir y verificar identidad del archivo.
- Breve análisis de por qué el algoritmo se comportó mejor o peor según el tipo de archivo.

Guión mínimo para la presentación

1. Presentar al grupo como proveedor y explicar las dos soluciones ofrecidas: tercerizada y propia.
2. Explicar brevemente el algoritmo propio y qué metadatos necesita para poder descomprimir.
3. Mostrar una compresión y una descompresión reales.
4. Demostrar integridad byte a byte / SHA-256.
5. Mostrar tabla comparativa propia vs. solución externa vs. gzip-6.
6. Interpretar ratio, ahorro, tiempos, throughput y Weissman Score.
7. Comparar el resultado propio con la tabla global y explicar en qué tipo de datos su algoritmo se desempeña mejor o peor.

FECHA DE ENTREGA: Martes 6 de Octubre

Nota metodológica sobre el Weissman Score

El Weissman Score combina ratio de compresión y tiempo normalizados contra una referencia. Fue creado por Tsachy Weissman y Vinith Misra a pedido de los productores de Silicon Valley. IEEE Spectrum ha señalado tanto su utilidad pedagógica como su sensibilidad a la unidad temporal y a la plataforma de ejecución. Por ese motivo este práctico fija unidad, baseline y protocolo de medición, y conserva métricas separadas de ratio y velocidad.

Referencias sugeridas: IEEE Spectrum, “A Fictional Compression Metric Moves Into the Real World” (2014) y “Weissman Score Calculators Migrate Online, But Metric Needs Some Improvements” (2017).

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026